CSC 2114 Assignment 3: Weighted Campus Route Finder (UCS and IDS)

Duration: 3 hours | **Language:** Python 3 (standard library only; `heapq` is allowed) | **Submission:** MUELE **Deliverables:** (1) `<group_number-Day A/B/Eve>_route_finder.py` (2) `<group_number-Day A/B/Eve>_report.pdf`

In Assignment 2 every walk cost was 1, so BFS was optimal by construction. In this assignment every walk has a real cost in metres, BFS stops being optimal, and you build the two algorithms that deal with that: **Uniform-Cost Search (UCS)** and **Iterative Deepening Search (IDS)**. You extend your own Assignment 2 `route_finder.py`; you do not start again.

1. The Problem

The visitor, the campus and the interface are the same as in Assignment 1: the user picks a **source place**, a **destination place** and a **search algorithm**, and the program prints the route it finds. Three things change.

1. **Edges now carry weights.** Each connection has a walking distance in metres (Section 2). The cost of a route is the **sum of its edge weights**, so the optimal route is the one with the smallest total distance. Hops are still counted and still reported, but they no longer define optimality.
2. **Two new algorithms.** The program supports `BFS`, `DFS`, `UCS` and `IDS`. UCS uses the weights. IDS, like BFS and DFS, does not.
3. **New instrumentation.** The program also reports how much memory each algorithm needed (maximum frontier size) and, for IDS, how its work is spread across depth limits.

What you must keep from Assignment 1: the `Node` class and `solution()`, the `reached` table, the single generic search loop, the goal test on removal from the frontier, the alphabetical neighbour ordering, the tree-like switch with the 10,000-expansion cap, and the input handling rules. Your Assignment 1 BFS and DFS routes must **not change**, because those two algorithms still ignore weights.

By the end you should be able to:

- explain why goal-testing on removal is what makes UCS optimal;
- explain why a cheaper path to a state can be found *after* the state was first reached, and how `reached` handles it;
- implement a priority-queue frontier and a depth-limited search without breaking the generic loop;

- compare four uninformed strategies on time, memory, route length and route cost, using your own measurements.

2. The Weighted Campus Graph (given)

The places and connections are exactly those of Assignment 2: 14 schools, 4 landmarks, 24 two-way edges. Each edge now has a weight, a **walking distance in whole metres**. Copy the list below into your program unchanged. As before, this is a simplified teaching model: the weights were chosen for this exercise and are **not survey-accurate**, so do not change them.

```
WEIGHTED_EDGES = [
    ("Main Building", "Freedom Square", 120),
    ("Main Building", "Main Library", 150),
    ("Main Building", "School of Social Sciences", 200),
    ("Main Building", "School of Liberal and Performing Arts", 350),
    ("Freedom Square", "Main Library", 90),
    ("Freedom Square", "School of Law", 180),
    ("Freedom Square", "Central Teaching Facility", 220),
    ("Main Library", "School of Education", 160),
    ("Main Library", "School of Economics", 400),
    ("Central Teaching Facility", "CoCIS", 130),
    ("Central Teaching Facility", "School of Statistics and Planning", 300),
    ("CoCIS", "School of Statistics and Planning", 100),
    ("CoCIS", "School of Engineering", 250),
    ("School of Engineering", "School of the Built Environment", 90),
    ("School of Engineering", "Margaret Trowell School of Industrial and Fine Art",
140),
    ("School of the Built Environment", "Margaret Trowell School of Industrial and Fine
Art", 70),
    ("Margaret Trowell School of Industrial and Fine Art", "School of Liberal and
Performing Arts", 450),
    ("School of Education", "School of Social Sciences", 140),
    ("School of Education", "College of Natural Sciences", 300),
    ("College of Natural Sciences", "School of Food Technology, Nutrition and
Bioengineering", 380),
    ("School of Food Technology, Nutrition and Bioengineering", "School of Agricultural
Sciences", 200),
    ("School of Agricultural Sciences", "School of Veterinary Medicine", 260),
    ("School of Statistics and Planning", "School of Economics", 350),
    ("School of Economics", "School of Law", 120),
]
```

Rules for using the weights

- Weights are positive integers and edges are symmetric: walking A to B costs the same as B to A. Integers keep path-cost comparisons exact, so ties are real ties and not rounding

accidents.

- The weights may be read **only** by the step-cost function, by UCS's priority queue, and by the code that reports a route's cost. They **must not** influence which node BFS, DFS or IDS chooses next.
- Your search code must receive the graph as a function argument and must not read the module-level list directly. Marking includes runs on a modified copy of the graph.
- The alphabetical neighbour rule from Assignment 1 still applies everywhere.

3. Practical Session Plan

Part	Task	Time
A	Formulate the weighted problem	30 min
B1	Graph, `Node`, `reached` updates	20 min
B2	Generic loop with a priority queue; UCS; IDS	45 min
B3	Interface, output and instrumentation	15 min
B4	Verify UCS and IDS behaviour	20 min
C	Experiments and results table	30 min
	Report writing	remaining time

Part A: Formulate (30 min)

Rewrite the five-component definition from Assignment 1 for the weighted problem. State the *type* of each component (set, function, Boolean test), and say which components changed and which did not.

1. **Initial state**
2. **Actions(s)**, including how the legal actions are read off the weighted graph
3. **Transition model** `Result(s, a)`
4. **Goal test**
5. **Path cost** and **step cost** `c(s, a, s')`. Give the step cost as a function of the graph, and state why it is positive.

Then answer each of the following in two or three sentences.

- **What the weights keep ignoring.** The weights in Section 2 are distances. List three real-world details of walking across campus that the model *still* ignores (for example slope,

crowds, rain). For one of them, say how it could be folded into the step cost without changing the *form* of the formulation.

- **Why positive step costs.** UCS needs every step cost to be at least some constant ε > 0. Describe what could go wrong with the search if some edge had weight 0, and what could go wrong if some edge had a negative weight.
- **State versus node, revisited.** In Assignment 1 you gave one state that appears in two nodes of the search tree. Now give an example from this graph of a state that appears in two nodes with **different path costs**, and say which of the two nodes UCS should keep.
- **What IDS needs from a node.** BFS and DFS used only `STATE`, `PARENT`, `ACTION` and `PATH-COST`. Which extra piece of information does a depth-limited search need, and how can a node obtain it from its parent?
- **Which quantity is minimised.** For each of BFS, UCS and IDS, state what quantity it minimises (hops, metres, or neither) when it returns a route.

Part B: Build and Search (100 min)

Extend your Assignment 1 `route_finder.py`. **Do not use `networkx` or any search or graph library.** The standard-library `heapq` module is allowed for the priority queue.

B1. Data structures (20 min)

- **Weighted graph.** Build an undirected adjacency structure from `WEIGHTED_EDGES`, for example `dict[str, dict[str, int]]`. Provide `neighbours(graph, s)` returning `(neighbour, weight)` pairs in **alphabetical order of the neighbour**, and `step_cost(graph, s, s2)`.
- **`Node`.** Keep the four lecture fields `STATE`, `PARENT`, `ACTION`, `PATH-COST`. `PATH-COST` is now the **sum of edge weights** from the root. Add a fifth field `DEPTH` (the number of hops from the root) and compute it from the parent's depth when the node is created. Hops reported to the user are `DEPTH` of the goal node.
- **`solution(node)`.** Unchanged: follow `PARENT` pointers back to the root and return the places in order.
- **`route_cost(graph, route)`.** A new independent checker that recomputes the cost of a list of places directly from the graph. Your program must use it in an assertion: the cost printed for every route has to equal `route_cost` of that route. This catches path-cost bugs early.
- **`reached`.** Still a dictionary from state to node, but its meaning has changed: it now holds the **cheapest node found so far** for each state, not merely the first one.

B2. Generic search loop (45 min)

Keep **one** search function that follows the lecture's seven steps. BFS, DFS and UCS differ only in the frontier discipline and in the rule for updating `reached`. Define a small frontier interface

(`push` , `pop` , `is_empty` , `__len__`) with three implementations and let the search function receive a frontier object; the loop itself must contain no `if algorithm == ...` branches.

Algorithm	Frontier	Remove	Add a child to the frontier when
BFS	queue (FIFO)	front	its state is not in `reached`
DFS	stack (LIFO)	back	its state is not in `reached`
UCS	priority queue (`heapq`) ordered by `PATH-COST`	lowest `PATH-COST`	its state is not in `reached` , **or** it reaches the state with a lower `PATH-COST` than the node stored in `reached`
IDS	stack, inside a driver loop (see below)	back	its state is not on the path from the root (cycle check)

Rules common to all four

- Apply the goal test **when a node is removed** from the frontier, never when it is generated. For UCS this is not optional: it is what makes the result optimal.
- **Nodes expanded** means nodes removed from the frontier that are not the goal and have their children generated, as in Assignment 1.
- Neighbours are considered in alphabetical order. For BFS, DFS and IDS the order of exploration is exactly as in Assignment 1: the alphabetically first neighbour is explored first, which for a stack means pushing children in reverse alphabetical order.
- The tree-like switch (`reached` off) and the 10,000-expansion cap work for BFS, DFS and UCS. When the cap is reached without finding the goal, stop and print `Search stopped: expansion cap (10000) reached.`

UCS details (these are marked)

- Each heap entry is the tuple `(path_cost, state_name, counter, node)` , where `counter` comes from `itertools.count()` . This gives a deterministic tie-break: lowest cost first, then alphabetically first state, then earliest inserted. It also stops Python from ever trying to compare two `Node` objects.
- A node can be in the heap after a cheaper node for the same state has replaced it in `reached` . When you remove such a **stale** node (its `PATH-COST` is greater than the cost stored in `reached` for its state), discard it silently. A discarded stale node is **not** counted as expanded.
- With `reached` off, UCS keeps every generated node and never discards one.

IDS details (these are marked)

- Write a driver `ids(graph, source, goal)` that calls the generic search with a stack frontier and `depth_limit = 0, 1, 2, ...`, with `reached` **off** and the **cycle check on**. The driver itself must not duplicate the loop.
- In an iteration with limit `L`, a node at depth `L` is still goal-tested when removed, but it is **not expanded** and does not count as expanded.
- The cycle check refuses to generate a child whose state already occurs on the path from the root to the parent (follow `PARENT` pointers). This is *not* the `reached` table: it remembers nothing about other branches.
- The search function must distinguish **cutoff** (the limit stopped some node from being expanded) from **failure** (nothing was cut off). If an iteration fails with no cutoff, the whole space has been explored: print `No route found.` Never let the limit exceed the number of places minus one.
- **Nodes expanded for IDS is the total over all iterations**, because that is the real work done. The route returned is the first goal found at the smallest limit, using the alphabetical order. IDS therefore returns a fewest-hops route; its cost in metres is whatever that route happens to cost.

B3. User interface (15 min)

Keep the interactive prompts for source, destination and algorithm. The algorithm prompt now accepts `BFS`, `DFS`, `UCS`, `IDS` and `ALL`, matched case-insensitively with surrounding spaces ignored. `ALL` runs the four algorithms in the order BFS, DFS, UCS, IDS and prints one block per algorithm, separated by a blank line.

The program must print each solution in exactly this format:

```
Algorithm: UCS
Route: <place 1> -> <place 2> -> ... -> <place n>
Cost (metres): <integer>
Hops: <integer>
Nodes expanded: <integer>
```

For IDS, print one extra line after `Nodes expanded:`:

```
Final depth limit: <integer>
```

Input handling (marked) follows Assignment 1: place names are matched case-insensitively and ignoring leading or trailing spaces; an unknown place or algorithm prints a clear message that lists the valid choices and does not crash; if source equals destination, print a route of one place with cost 0, hops 0 and nodes expanded 0; if no route exists, print `No route found.`

New command-line flags

Flag	Effect
`--no-reached`	Tree-like search for BFS, DFS and UCS, with the 10,000 cap.
`--trace`	For UCS, print each frontier push, each removal, each discarded stale node, and each `reached` replacement, one line per event.
`--verbose`	After the normal output, also print `Max frontier size: <integer>` (the largest number of nodes the frontier held at any moment). For IDS also print one line per iteration: `limit <L>: expanded <integer>`.

B4. Verify UCS and IDS (20 min)

Run the checks below and record what you observe in the report.

1. **Check your numbers.** The table gives the correct results for three pairs. If yours differ, find the bug before going further.

Source	Destination	UCS cost (m)	UCS hops	UCS expanded	IDS cost (m)	IDS hops	IDS expanded (total)
School of Education	School of Economics	550	4	8	560	2	4
CoCIS	School of Liberal and Performing Arts	820	4	13	840	3	11
School of the Built Environment	School of Law	870	5	11	1170	5	32

2. **Watch `reached` being updated.** Run UCS with `--trace` from **School of Law to School of the Built Environment**. Find the line where a state is first reached at one cost and later replaced by a cheaper node. Paste the relevant trace lines into the report and explain, in three or four sentences, what happened and why the first node was not good enough.
3. **Tree-like UCS.** Run UCS with `--no-reached` between **School of Education and School of Economics**, then between **School of Statistics and Planning and School of Veterinary Medicine**. Record nodes expanded for both, and compare with the graph-search counts. One of the two runs should hit your cap. Explain in the report why cycles cause this, and

why UCS in tree-like mode would still stop eventually if the cap were removed and the goal is reachable.

4. **IDS without `reached`.** IDS also runs without a `reached` table, yet it terminates and never loops. Explain in two or three sentences which mechanism makes that true, and what IDS gives up by not having `reached`.

Part C: Experiments and results (30 min)

Choose **ten distinct pairs of schools** (the pair A to B and the pair B to A count as the same pair). Run **all four** algorithms on each pair, using graph search with `reached` on and `ALL` mode. Report the results in one table with forty rows:

#	Source	Destination	Algorithm	Route	Cost (m)	Hops	Nodes expanded

Your ten pairs, taken together, must satisfy **every** condition below. Name the row numbers that satisfy each condition in a line under the table.

Condition	How many
UCS returns a route with a **lower cost** than BFS	at least 3 pairs
UCS's route has **more hops** than BFS's route (cheaper in metres, longer in hops)	at least 1 pair
IDS returns a route that costs **more metres** than UCS's	at least 2 pairs
UCS expands **more** nodes than BFS	at least 1 pair
UCS expands **fewer** nodes than BFS	at least 1 pair
IDS expands at least **five times** as many nodes as BFS	at least 1 pair
The two schools are **directly connected**	at least 1 pair
The fewest-hops distance between the two schools is **six or more**	at least 1 pair

Also run `--verbose` on the **deepest pair in your table** (the one with the most hops) for all four algorithms, and record `Max frontier size` for each, plus IDS's per-iteration counts. You will need them in the analysis.

Check before you submit: every cost in your table must equal what `route_cost` computes for the route shown, and your BFS and DFS routes must match your Assignment 1 results for the

same pairs.

4. Report (max 5 pages, PDF)

The 40-row results table and the pasted `--trace` lines may go in an appendix that does not count toward the five pages. Everything else must fit inside them.

1. **Problem formulation.** Your Part A answers.
2. **Design.** A short description of what changed since Assignment 1: `Node` (with `DEPTH`), the `reached` update rule, the three frontier classes, the UCS tie-break and stale-node rule, and the IDS driver. Include pseudocode or a diagram of the generic search loop that shows where the frontier, the `reached` rule and the depth limit plug in.
3. **Verification (B4).** Your check against the three reference rows, the `--trace` excerpt with its explanation, the tree-like UCS counts with their explanation, and your answer on IDS without `reached`.
4. **Results table (Part C).** With the condition lines, the `--verbose` measurements for your deepest pair, and one pasted terminal run of `ALL`.
5. **Analysis.** This is the substantive part. Answer every question below, and support each answer with specific rows or numbers from *your own* results.
    1. **UCS optimality.** State the two conditions from the lecture that make UCS return a cheapest route (one concerns the step costs, one concerns when the goal test happens). Then pick a pair from your table where UCS is cheaper than BFS, and show by hand what a UCS that goal-tested **at generation** would have returned instead.
    2. **BFS stops being optimal.** This answers last week's closing question. Pick a pair where BFS's route is not the cheapest. Draw both routes on the weighted graph with the edge weights, give both costs, and explain why BFS, which minimises hops, cannot see the difference.
    3. **IDS versus BFS and UCS.** IDS and BFS both return a fewest-hops route. Is IDS ever cheaper in metres than UCS? Use your table to say when IDS's cost equals UCS's cost and when it does not.
    4. **Where IDS's repeated work goes.** Using your `--verbose` per-iteration counts for the deepest pair, give the expansions at each limit and the total. Compare the total with BFS's expansions for the same pair. The lecture argues that IDS's repeated work is small. Explain why that argument holds for trees with a large branching factor, and why it does *not* hold well on this small, heavily cyclic graph.
    5. **Memory.** Using your `Max frontier size` measurements for the deepest pair, rank BFS, DFS, UCS and IDS by memory and explain the ranking. Say what in this graph makes the BFS and UCS frontiers grow, and what the lecture says about the memory needed by IDS compared with BFS.

6. **Two ways to avoid loops.** `reached` and IDS's cycle check both stop the search from going round cycles. Compare them: how much memory does each use, which one lets UCS reopen a state at a lower cost, and what could each wrongly throw away?
7. **Do the uninformed algorithms read the weights?** Suppose the edge between Margaret Trowell School of Industrial and Fine Art and School of Liberal and Performing Arts dropped from 450 m to 150 m. Without running anything, say which of BFS, DFS, UCS and IDS could return a different **route**, which could return a different **cost** for the same route, and which would be unaffected. Then check your prediction by running it and report the outcome.
8. **Limitation of UCS.** UCS expands every place cheaper than the goal, including places that lie in the opposite direction. Using one pair from your results, give the number of expansions that moved away from the destination. Name the algorithm from the lecture's best-first table that would avoid this, and say what extra information it needs. You do not need to implement it.

6. **Reflection.** In one short paragraph: what one thing would you change about the weighted campus model to make it more realistic (for example, weights that depend on direction, on time of day or on the walker), and which of the five problem components would that change?

5. Marking Scheme (100 marks)

Component	Marks
Part A: five-component formulation with weights, step cost, and the five short answers	12
Weighted graph, `Node` with `DEPTH`, `solution()`, `route_cost()` check, `reached` as cheapest-node table	12
One generic search loop; BFS, DFS and UCS differ only in frontier and `reached` rule; IDS built as a driver over the same loop	20
Correct routes, costs and expansion counts for all four algorithms (alphabetical ordering, UCS tie-break, stale-node rule, IDS total expansions)	20
Interface, output format, `ALL` mode, flags, and input error handling	8
Verification (B4): trace, tree-like UCS counts, explanations	7
Results table meets all Part C conditions	5
Analysis quality and use of evidence	13
Code clarity, comments, and the report's presentation	3

Automatic deductions

- Hard-coded routes: 0 for the correctness marks.
- Use of a search or graph library: −20.
- Edge weights used in any decision made by BFS, DFS or IDS (other than reporting cost): −10.
- Search functions that read the module-level edge list instead of the graph argument: −5.
- Unresolved plagiarism referral.

6. Notes

- **Every hop no longer costs 1.** Cost is measured in metres. UCS's route is never costlier in metres than any other algorithm's route. BFS's and IDS's routes are never longer in hops than any other algorithm's route, but they can cost more in metres.
- **Counts are marked exactly.** Use the node-expansion definition in B2: the goal node is not counted, stale nodes are not counted, and IDS counts expansions across all iterations.
- **Reuse your Assignment 2 code,** but submit one program that does all four algorithms. Keep it readable: someone should be able to find the `Node` class, the frontier classes, the generic search loop and the IDS driver in under a minute.

- Submit only `the python file report pdf file using the naming convention given above.

Before next week: read Russell & Norvig 4th ed., section 3.5 (informed search). Look at the "Nodes expanded" column for UCS in your table and find the pair where UCS spent the most expansions on places in the wrong direction. What would you need to know about the destination to avoid them, and where in this model could that information come from?